

SWE 4101

Object Oriented Concepts I

Prepared By:
Samia Nawsheen
Junior Lecturer, CSE

Objectives

01 Recap

02 A New Problem

03 What an Interface is

04 Implementing & Subtyping

05 Many Interfaces, One Class

06 Interface vs Abstract Class

Recap: Where We Left Off

- Abstract Employee class
- Manager, Developer, Salesperson extended Employee
- Each subclass overrides calculatePay()
- We could have a polymorphic loop where each type of employee runs its own version of the method

A New Problem

- Accounts must pay all employees at the end of the month
- Contractors and vendors also need to be paid
- How do we pay everyone in one monthly run?
- A single pass over a mixed group needs one common type to loop over
- A Contractor is not an Employee; a Vendor isn't even a person
- Forcing "extends Employee" would be an is-a lie

Question for the class

*Employee, Contractor, and Vendor share no honest is-a relationship.
What do these three actually have in common?*

Working it through

- They share a capability, not an identity
- Each one can be asked: "how much do we owe you?"
- No shared attributes but have a common capability – getting paid
- That capability alone is what an interface captures

What is an Interface

- A contract: method signatures a class promises to implement
- No fields, no constructor
- Methods are implicitly public and abstract
- It is a type – usable as a variable, parameter, or return type

Declaring an Interface

```
public interface Payable {  
    long amountOwedThisMonth();  
}
```

- Just a name and a set of promises
- No method body — the signature ends with a semicolon
- Cannot be instantiated: `new Payable()` is illegal

Common mistake

```
Payable p = new Payable();  
// Compile error!
```

- An interface is a contract, not an entity — nothing to construct
- You may only instantiate a concrete class that implements it
- Store that object in a Payable-typed variable instead

Implementing an Interface

```
public class Employee implements Payable {  
    @Override  
    public long amountOwedThisMonth() {  
        return calculatePay();  
    }  
}
```

- implements signs the contract
- Must provide a body for every promised method
- The class keeps its own unrelated fields and code

Unrelated Classes, Same Contract

```
public class Contractor implements Payable {  
    private long ratePerMilestone;  
    private int milestonesSignedOff;  
  
    public long amountOwedThisMonth() {  
        return ratePerMilestone * milestonesSignedOff;  
    }  
}
```

- Contractor shares no field or code with Employee
- Only the method signature is shared
- Vendor would look just as unrelated, with its own fields

Interfaces Create Subtypes Too

```
Payable[] toPay = {  
    new Employee(...), new Contractor(...), new Vendor(...)  
};  
  
for (Payable p : toPay) {  
    total += p.amountOwedThisMonth();  
}
```

- One variable type, many unrelated implementers
- Compiler checks only that Payable promises the call
- JVM dispatches to each object's actual class at runtime

Question for the class

Payable itself has no body for `amountOwedThisMonth()`. Why does `p.amountOwedThisMonth()` still compile — and why does it run the right formula for each object?

Working it through

- Compile-time: the compiler checks only the declared type, Payable
- Runtime: the JVM checks the actual object's real class
- The same rule as method overriding
- Dispatch doesn't care whether the promise came from extends or implements

One Class, Many Interfaces

```
public class Contractor implements Payable, Taxable, Auditable
{
    // must implement all three contracts
}
```

- A class can implement as many interfaces as it needs
- But it can extend only one superclass

Question for the class

*A class may implement many interfaces, but may extend only one class.
Why is stacking many contracts safe, when stacking many parent classes
is not?*

Working it through

- Two parent classes could define the same field or method — ambiguous
- That's the classic "diamond" problem
- Interfaces carry no state or bodies (traditionally) — nothing to collide
- Many promises compose cleanly; many implementations do not

Many Views, One Object

```
Contractor c = new Contractor(...);  
Payable    asPayable    = c;  
Taxable    asTaxable    = c;  
Auditable  asAuditable  = c;
```

- Same object, three different role-typed views
- Each reference exposes only that role's promised methods

Interfaces Cut Across Hierarchies

- Employee, Contractor, and Vendor share no family tree
- Yet all three are Payable
- Inheritance groups objects by what they are
- Interfaces group objects by what they can do

Choosing the Right Tool

- "B is a A" — shared identity and code -> extends
- "B can do A" — shared ability only -> implements
- "B has a A" — composition -> a field
- Interface names often end in -able: Payable, Comparable, Runnable

Common mistakes

- Naming an interface after a noun/thing: interface Animal
- Naming an abstract class after an ability: abstract class Comparable
- Ask first: do they share fields and code, or just one ability?

Interface vs Abstract Class

Feature	Abstract class	Interface
Keyword	extends	implements
How many?	One parent only	Many interfaces
Instance fields	Yes	No — only constants
Constructor	Yes	No
Method bodies	Yes, freely	No (exceptions exist)
Models	"is a"	"is able to / acts as"

Practice exercise

- 1 Take the Shape family (Circle, Square, Triangle) from the last lab
- 2 New requirement: "anything that can be saved to disk"
- 3 Decide: does this belong in the class hierarchy, or a new interface?
- 4 Name the interface as an ability
- 5 Find one unrelated, non-shape class that could honestly implement it

Thank You